mini-ddpm: MNIST diffusion from scratch

Fazal Hameed | Experimental report | 18 October 2026

Method

A miniature unconditional DDPM is trained from random initialization to predict Gaussian noise on 28 x 28 MNIST. The forward process, time-conditioned U-Net, loss and reverse sampler are implemented with basic PyTorch; no pretrained generator or diffusion framework is used. The linear schedule has T = 1,000, beta from 0.0001 to 0.02, and alpha_bar(T) = 0.00004036 [1].

```
alpha_t = 1 - beta_t; alpha_bar_t = product(alpha_1 ... alpha_t)
x_t = sqrt(alpha_bar_t) x_0 + sqrt(1 - alpha_bar_t) epsilon
L = E[ || epsilon - epsilon_theta(x_t,t) ||^2 ], epsilon ~ N(0,I)
```

The U-Net uses resolutions 28, 14 and 7, channel widths 32/64/64, six residual blocks, GroupNorm, SiLU, sinusoidal timestep embeddings, and encoder-decoder skip connections. Ancestral sampling starts at Gaussian noise, predicts and clips x_0 to [-1,1], and uses the fixed-small Gaussian posterior variance. The final clean transition adds no noise.

Training setup and recorded outcome

Item	Recorded setting
Data / scaling	55,000 train + 5,000 validation; fixed split seed 31415; pixels / 127.5 - 1
Capacity / optimization	550,465 parameters; AdamW, LR 0.0002, weight decay 0.0001; gradient clip 1.0
Budget / weights	12 epochs; batch 256; 2,580 updates; seed 42; final EMA, decay 0.995
Hardware / software	Free Tesla T4; PyTorch 2.11.0+cu130; Python 3.13.15; two CPU threads
Precision	FP32 parameters; FP16 convolution/linear autocast; gradient scaling, initial scale 1,024
Measured training	138.16 s (2.30 min), synchronized epoch fit loops; excludes validation/checkpoint I/O
Final loss	Online training MSE 0.02628; EMA validation MSE 0.02591

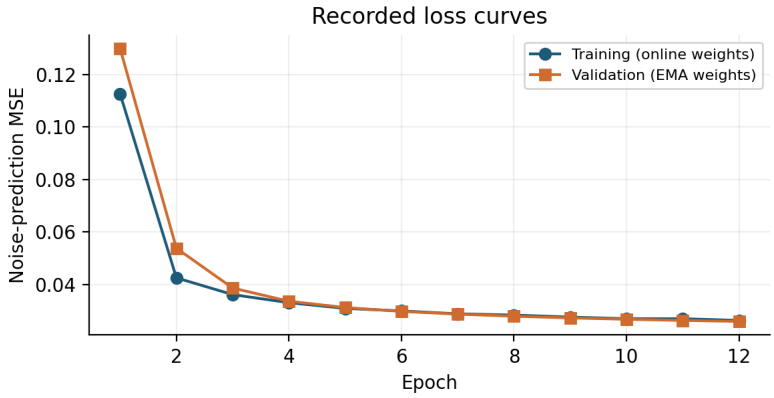


Figure 1. Full epoch history; validation uses fixed noise/timesteps. Online training and EMA validation weights differ.

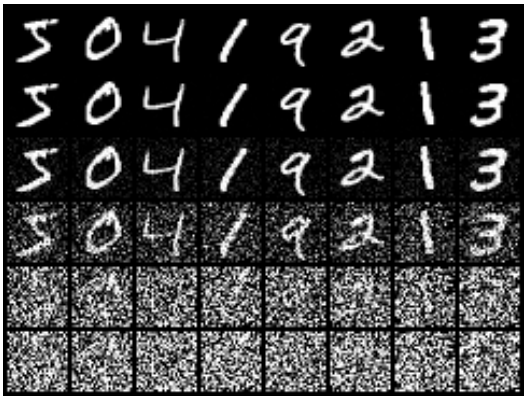


Figure 2. Eight input digits (top), then noising at original steps 1, 50, 200, 500 and 1,000. Rows share the same epsilon.

Required 8 x 8 ancestral sample grids

The first 64 of 512 generated images are shown for each setting, without quality filtering. All settings use the same final EMA checkpoint and sampling seed 2026. Initial noise is shared; intermediate random paths differ between chain lengths.

DDPM: 1,000 steps



DDPM: 200 steps



DDPM: 50 steps



Runtime and quantitative diagnostics

Sampler	Steps	64 images, seconds median [min, max]	Speedup	MNIST feature distance (lower)	Confidence >= 0.9 (%)
DDPM	1000	5.124 [5.085, 5.219]	1.00x	30.64	53.71
DDPM	200	1.016 [1.012, 1.059]	5.04x	36.98	52.15
DDPM	50	0.257 [0.253, 0.259]	19.91x	37.90	47.66
DDIM (bonus)	50	0.266 [0.256, 0.269]	19.23x	38.95	43.55

Runtime: three rotated-order repeats with batch 64 after kernel warm-up; CUDA is synchronized around generation. Timing includes random draws and denoising, and excludes loading, plotting, metrics and file I/O. Quality uses a separate fixed 512-image batch per setting. The report makes no laptop CPU runtime claim.

Evaluator: a separate CNN trained from scratch for three epochs reaches 98.09% accuracy on the 10,000-image MNIST test set. Feature distance compares mean/covariance of its 64-dimensional penultimate features against 2,048 fixed test images. This is **not standard Inception FID**. A disjoint real-512 versus reference calibration scores 6.67. Confidence on unlabeled generated images is a diagnostic, not classification accuracy.

What the comparison shows

Many samples have recognizable digit shapes, but every grid also contains ambiguous or malformed digits. In this run, 1,000-step DDPM has the lowest measured feature distance. At 200 steps, generation is 5.04x faster and the high-confidence fraction is 52.15% versus 53.71%. At 50 steps, speed rises to 19.91x, with worse feature distance and a lower high-confidence fraction. All settings receive predictions spanning all ten classes; predicted-class entropy is 0.979-0.992 of the maximum. These finite-sample diagnostics do not establish a universal ranking.

Optional bonus: deterministic DDIM, 50 steps



DDIM uses the same trained noise predictor and time indices, with eta = 0 and no injected reverse-process noise [2]. Epsilon is recomputed after clipping the predicted x_0 to keep the decomposition consistent. The 8 x 8 grid again contains the first 64 unfiltered samples.

Its median batch-64 time is 0.266 s. Feature distance is 38.95 and the high-confidence fraction is 43.55%. The bonus works, but does not improve quality over 50-step ancestral DDPM in this measured run. No class conditioning is implemented.

Implementation issues and solutions

Short chains and noise range. Simply retaining the same beta endpoints in a new 50-step chain leaves alpha_bar(T) near 0.60, inconsistent with a standard-Gaussian start. The 200/50-step samplers retain the original alpha-bar marginals and recompute the Gaussian posterior for each selected jump [3].

Indices, clipping and numerical stability. The code uses indices 0-999 and s = -1 for the clean image. Schedule products use FP64; image computation uses FP32. Predicted x_0 is clipped and posterior variances are clamped against rounding error. Tests check adjacent-posterior equivalence and the noiseless final step.

Live execution logs. The first evaluation was interrupted while diagnosing missing notebook subprocess output. Completed generator and classifier weights were preserved. Final comparisons were rerun with streamed logs and the same fixed checkpoints; partial interrupted timings are not used.

Reproducibility and limits

The submission includes runnable PyTorch source, EMA generator and evaluator weights, exact setup, losses, all generated batches, timing repetitions, and seven passing equation/CPU tests. A CPU sampling smoke test also loads the supplied weights. MNIST gzip checksums and source hashes are retained. The code supports CPU training and sampling; full CPU training time was not measured. One generator seed, 512 samples per setting and an imperfect digit classifier limit quality conclusions. More training, independent sample seeds and blinded human judgments could strengthen evaluation.

Run:
```
python -m pip install -r requirements.txt
python test_ddpm.py
python sample.py --device cpu --steps 200
python mini_ddpm.py train --device cpu
```

References

[1] Ho, Jain and Abbeel (2020), DDPM. arxiv.org/abs/2006.11239

[2] Song, Meng and Ermon (2020), DDIM. arxiv.org/abs/2010.02502

[3] Respacing reference: OpenAI improved-diffusion, respace.py. MNIST: LeCun, Cortes and Burges; mirror/checksums documented in Torchvision datasets/mnist.py.